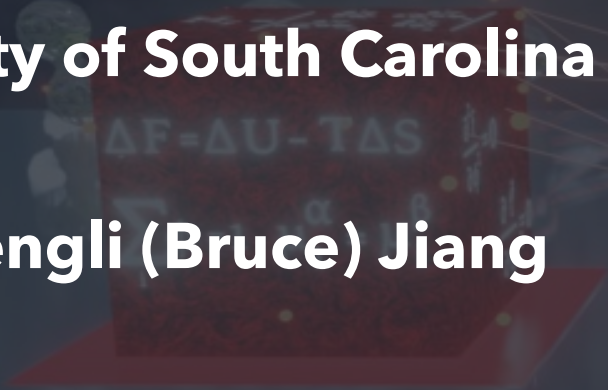


Regularization and Hyperparameter Tuning

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang

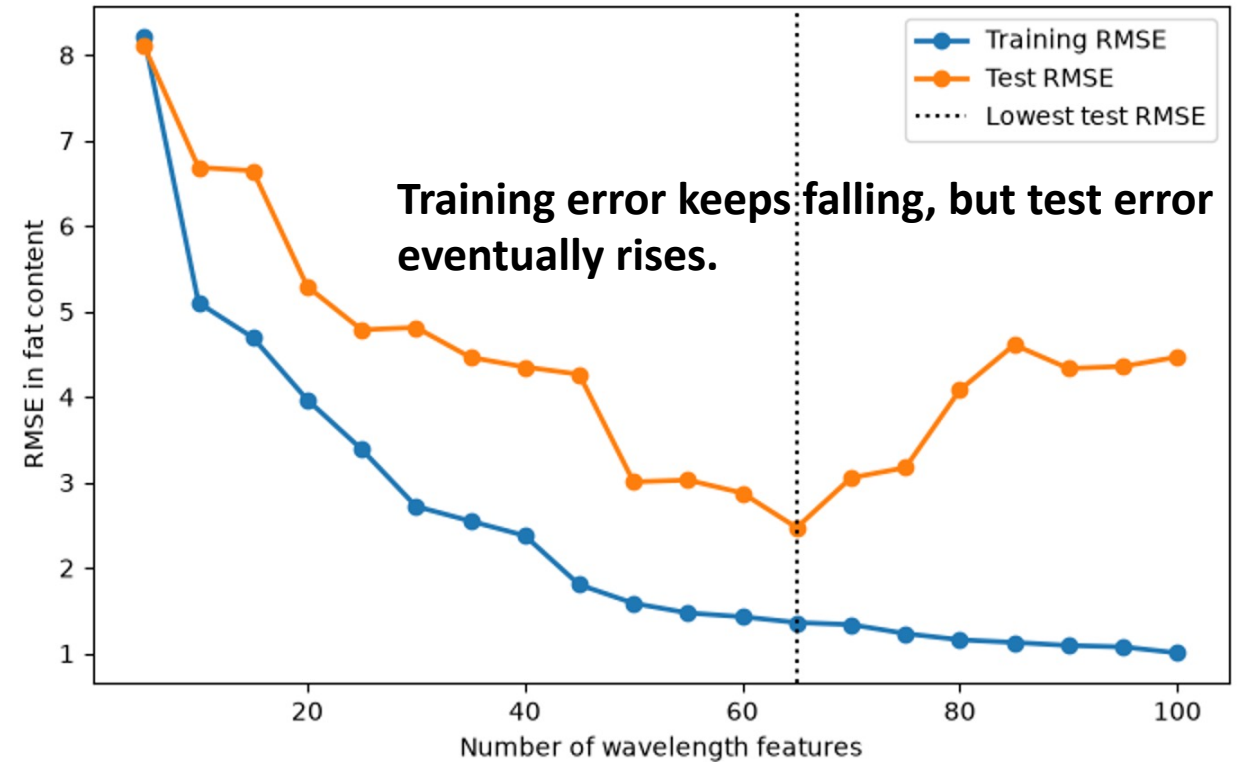
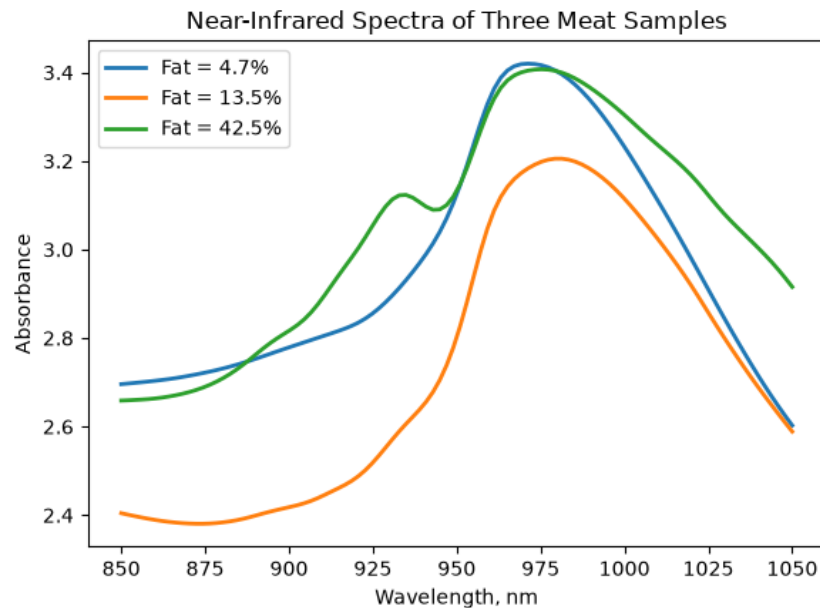


More Features Can Hurt Test Performance

Tecator meat spectroscopy dataset

<https://www.openml.org/search?type=data&id=505&sort=runs&status=active>

The Tecator dataset contains near-infrared spectra from 240 meat samples. Each sample has absorbance measured at 100 wavelengths from 850 to 1050 nm, and the goal is to predict its fat content.



```
for count in feature_counts:
    model = LinearRegression()
    model.fit(X_train[:, :count], y_train)

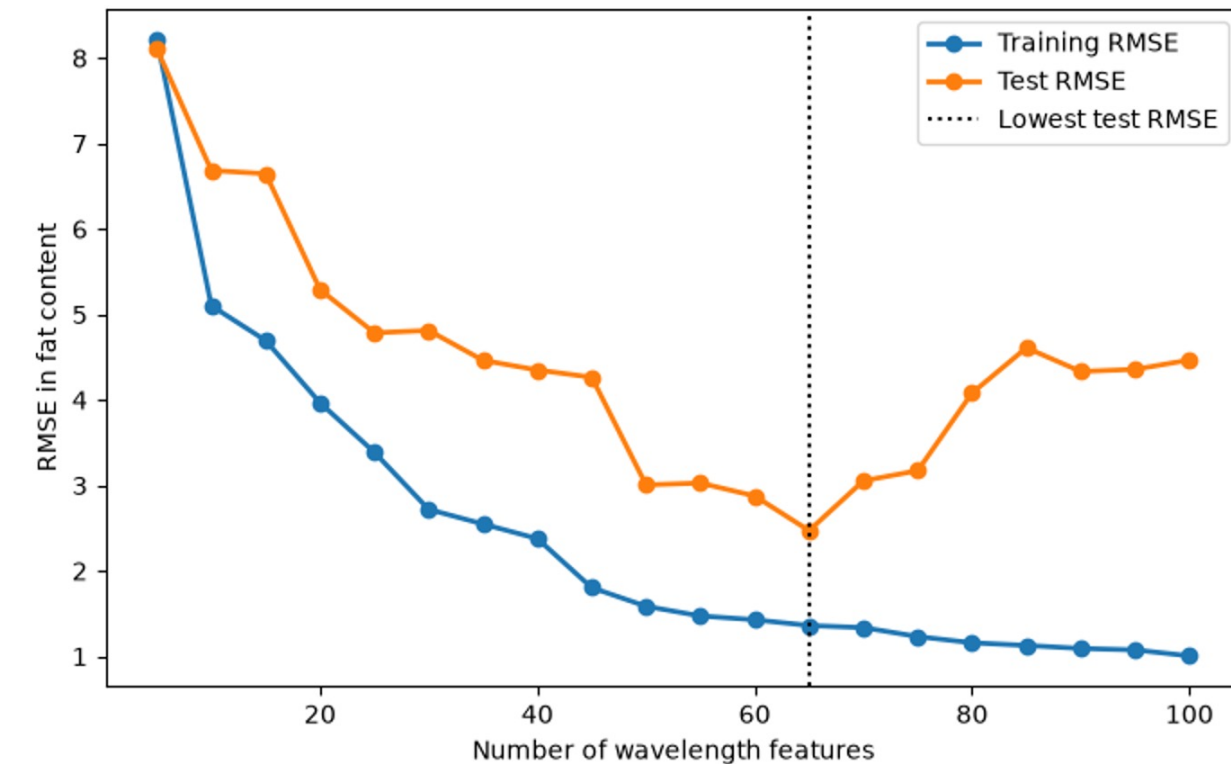
    y_train_prediction = model.predict(X_train[:, :count])
    y_test_prediction = model.predict(X_test[:, :count])

    train_rmse.append(np.sqrt(mean_squared_error(y_train, y_train_prediction)))
    test_rmse.append(np.sqrt(mean_squared_error(y_test, y_test_prediction)))
```

Training : Test = 60% : 40%

100 features means 100 adjustable slopes fitted from only 144 training samples.

Extra Features Give the Model More Ways to Fit Noise



```
for count in feature_counts:
    model = LinearRegression()
    model.fit(X_train[:, :count], y_train)

    y_train_prediction = model.predict(X_train[:, :count])
    y_test_prediction = model.predict(X_test[:, :count])

    train_rmse.append(np.sqrt(mean_squared_error(y_train, y_train_prediction)))
    test_rmse.append(np.sqrt(mean_squared_error(y_test, y_test_prediction)))
```

- 10 wavelength features -> 10 slope parameters
- 60 wavelength features -> 60 slope parameters
- 100 wavelength features -> 100 slope parameters

A new feature **cannot** make the best training fit worse. Why?

$$f(\mathbf{x}_i) = \theta_1 x_{i1} + \theta_2 x_{i2} + \cdots + \theta_p x_{ip}$$

The model can always set its parameter to **zero**.

- **Useful features:** adds new chemical information (*a new adsorption peak*)
- **Redundant features:** adds another parameter but little new information (*repeats a neighboring wavelength*)
- **Too much freedom:** allows the model to reproduce training-specific noise

The model **overfits** when extra parameters explain accidental details that do not repeat in new samples.

Correlated Features Can Produce Similar Predictions

Consider ordinary least squares

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta}$$

$$\hat{\boldsymbol{\theta}}_{\text{OLS}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2$$

Create two highly correlated features:

$$x_{i2} = x_{i1} + \text{small noise}$$

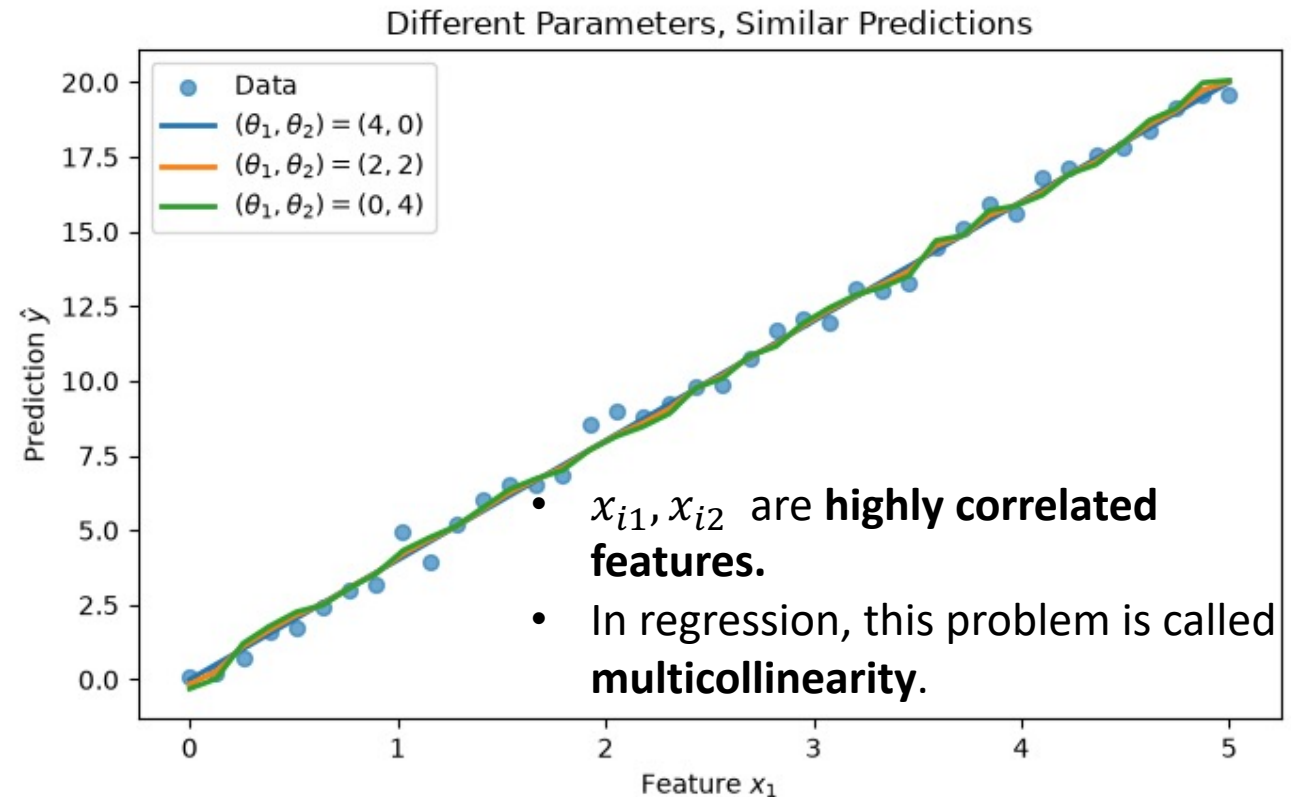
Suppose the output mainly depends on their sum:

$$y_i \approx 4x_{i1}$$

What could be the possible solutions?

$$(\theta_1, \theta_2) = (4, 0), \quad (2, 2), \quad (0, 4)$$

Because $x_{i1} \approx x_{i2}$, these parameter pairs can produce similar predictions!

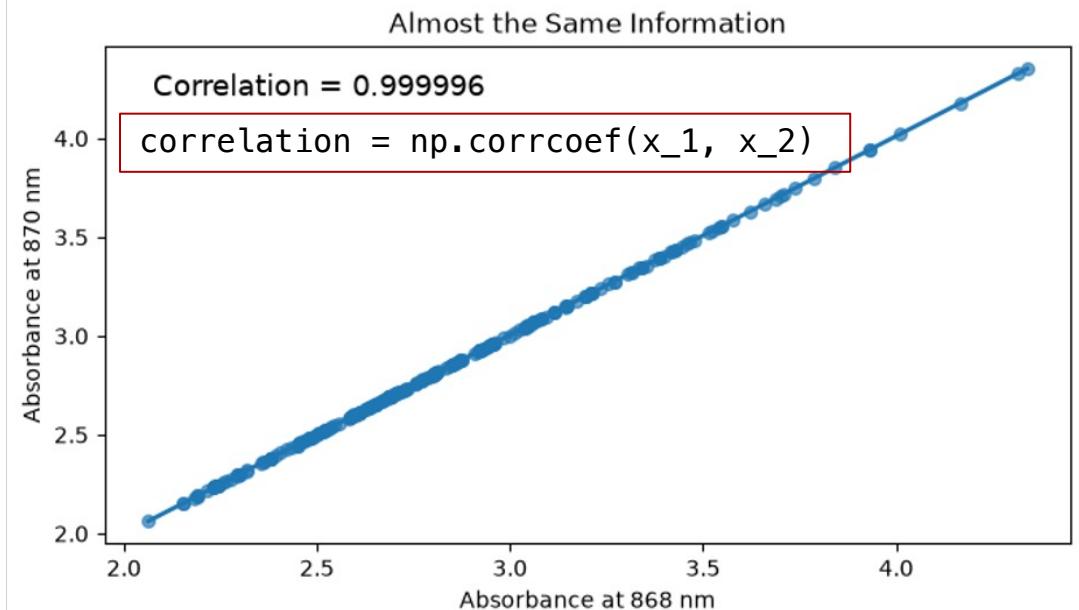
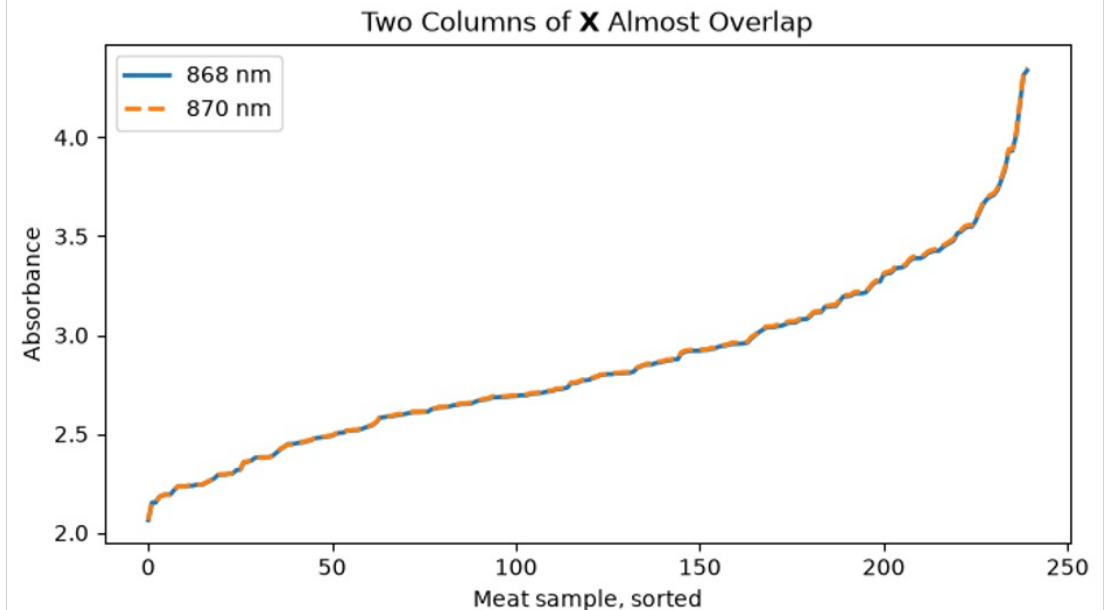
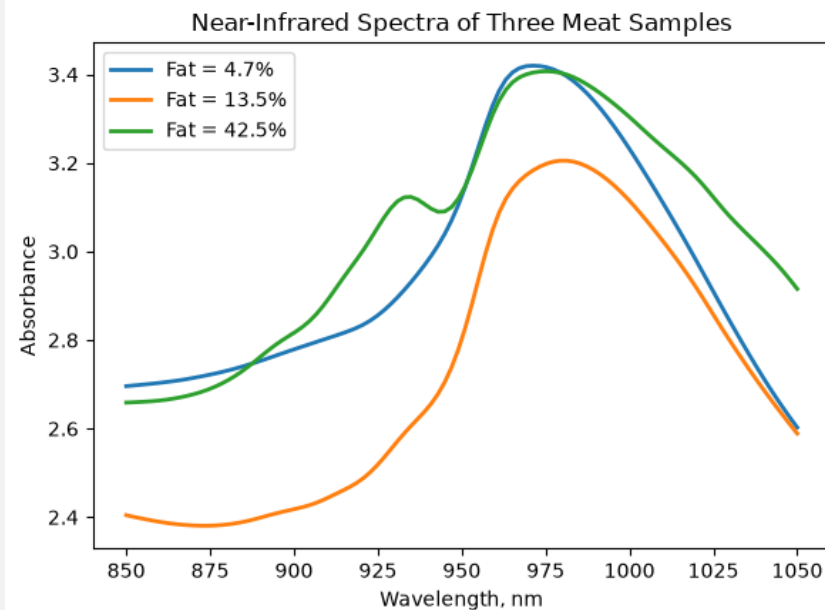


Correlation Makes Individual Parameters Unstable

Tecator meat spectroscopy dataset

<https://www.openml.org/search?type=data&id=505&sort=runs&status=active>

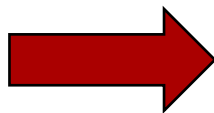
The Tecator dataset contains near-infrared spectra from 240 meat samples. Each sample has absorbance measured at 100 wavelengths from 850 to 1050 nm, and the goal is to predict its fat content.



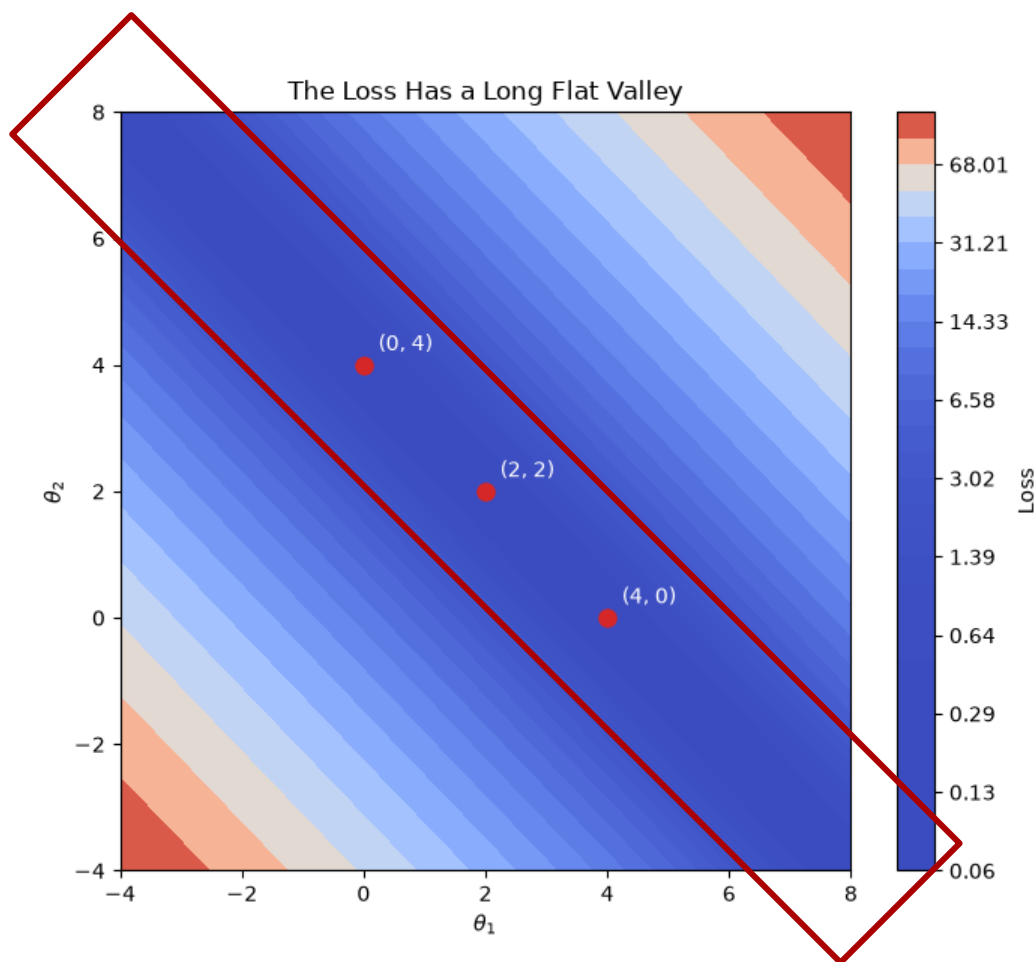
Correlation Creates a Long, Flat Loss Valley

The squared-error loss is:

$$J_{\text{data}}(\boldsymbol{\theta}) = \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2$$



- The pairs (4,0), (2,2), and (0,4) produce almost the same predictions.
- A small change in the data can move the fitted parameters far along the valley.



Predictions can remain similar while parameter values change dramatically.

Can we constrain/regularize the parameters?

Regularization Selects a More Controlled Solution

Regularized regression minimizes two quantities:

$$J(\boldsymbol{\theta}) = J_{\text{data}}(\boldsymbol{\theta}) + \lambda P(\boldsymbol{\theta})$$

where:

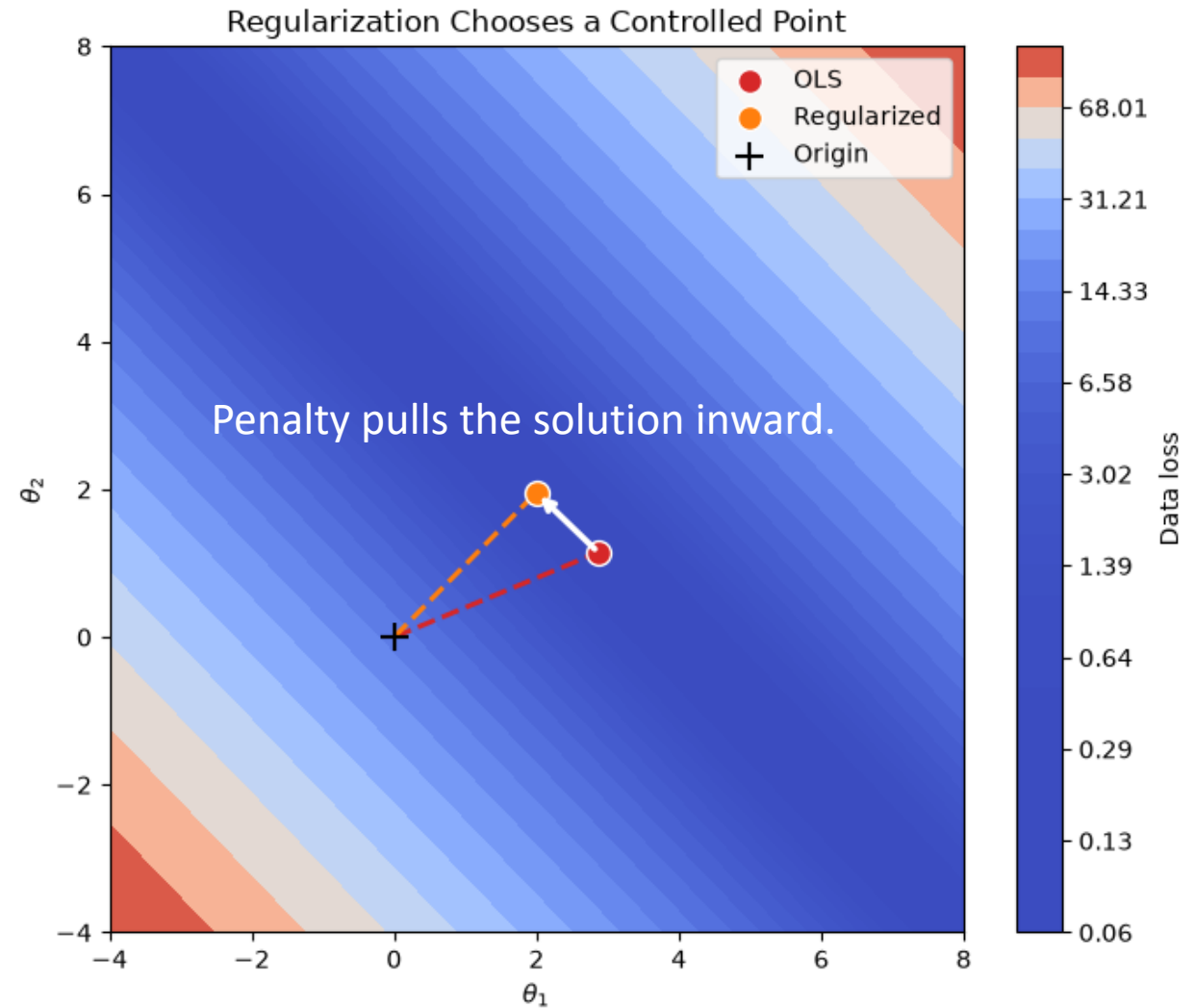
J_{data} = prediction error

$P(\boldsymbol{\theta})$ = penalty on parameter size

Two types of regularization:

Ridge $P(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_2^2 = \sum_{j=1}^p \theta_j^2$

Lasso $P(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_1 = \sum_{j=1}^p |\theta_j|$



The **data** determine the valley. The **penalty** tells the model which point inside that valley we prefer.

Ridge Makes Large Parameters Expensive

$$\hat{\boldsymbol{\theta}}_{\text{ridge}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \left[\frac{1}{2n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 + \frac{\lambda}{2} \|\boldsymbol{\theta}\|_2^2 \right]$$

$$\|\boldsymbol{\theta}\|_2^2 = \sum_{j=1}^p \theta_j^2 \quad \Rightarrow \quad 1^2 = 1, \quad 5^2 = 25 \quad \Rightarrow \quad \text{A parameter five times larger receives a penalty twenty-five times larger.}$$

- Large parameters become expensive.
- Ridge pulls every parameter toward zero.
- Parameters usually remain nonzero.
- Larger λ means stronger shrinkage.

Ridge accepts a slightly worse training fit to obtain smaller, more stable parameters.

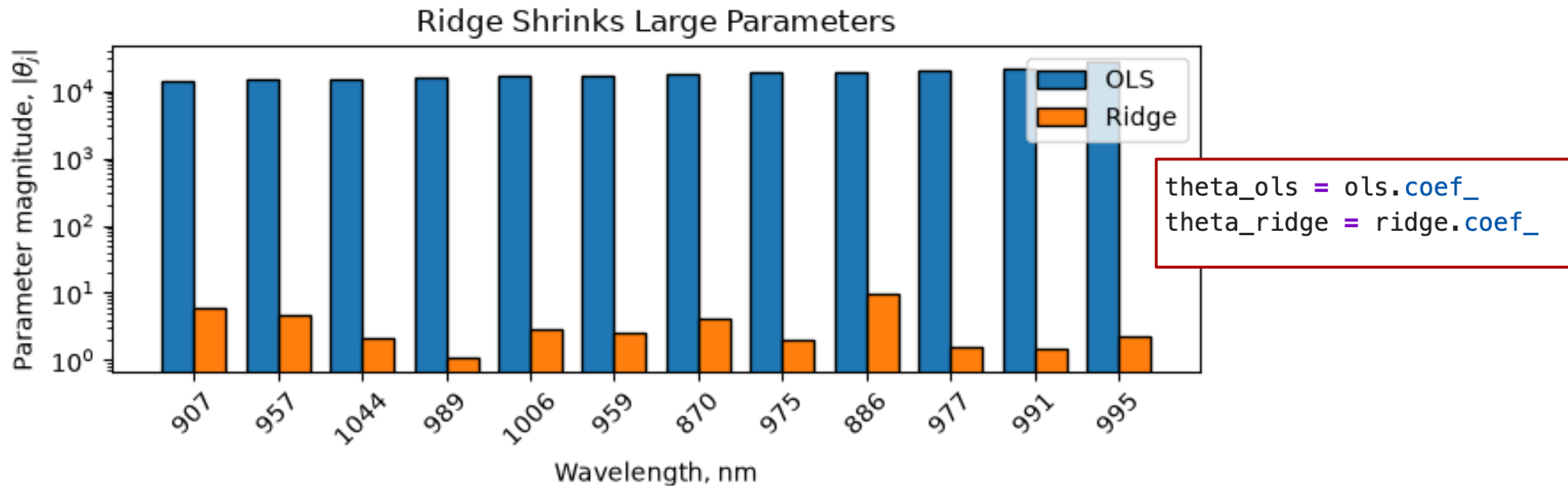
```
class sklearn.linear_model.Ridge(alpha=1.0, *, fit_intercept=True,  
copy_X=True, max_iter=None, tol=0.0001, solver='auto', positive=False,  
random_state=None)
```

```
data = pd.read_csv("tecator.csv")  
  
feature_columns = [f"absorbance_{i}" for i in range(1, 101)]  
  
X = data[feature_columns]  
y = data["fat"]  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.40, random_state=26  
)  
  
ols = LinearRegression().fit(X_train, y_train)  
ridge = Ridge(alpha=.1).fit(X_train, y_train)
```

Ridge Trades Training Fit for Better Test Performance

Model	Training RMSE	Test RMSE
OLS	1.00	4.47
Ridge	3.24	4.25

In this split, Ridge accepts a worse training fit but performs slightly better on **unseen** samples.



But what if the original features have different scales?

Standardization Makes Parameter Penalties Comparable

$$\|\boldsymbol{\theta}\|_2^2 = \sum_{j=1}^p \theta_j^2$$

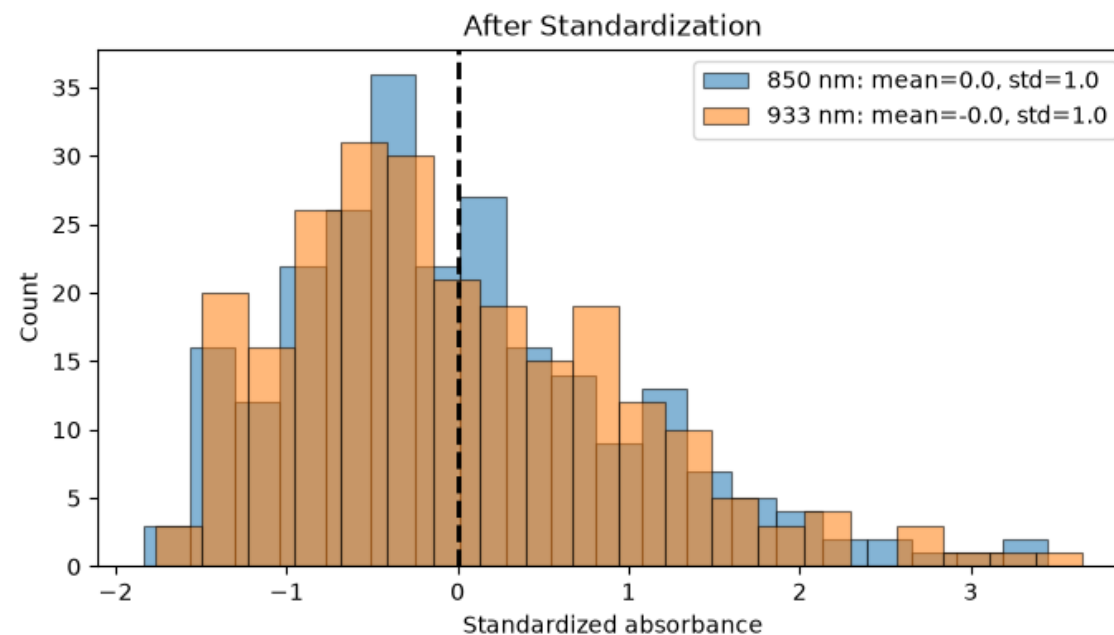
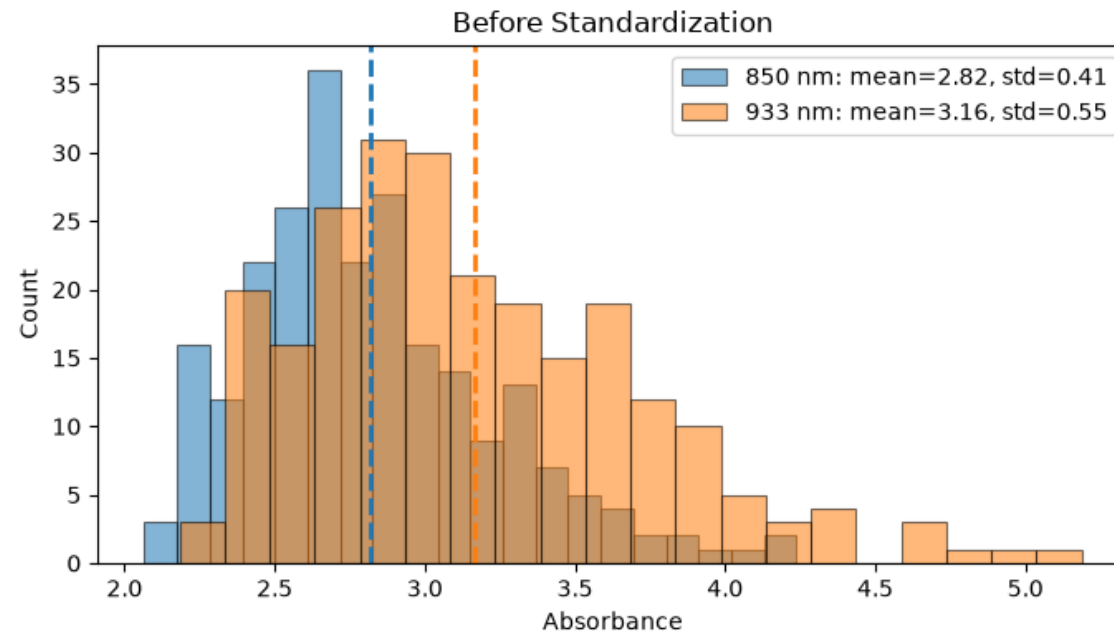
A penalty compares parameter sizes, so the features must first be placed on comparable scales.

Standardize every **feature**:

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$$

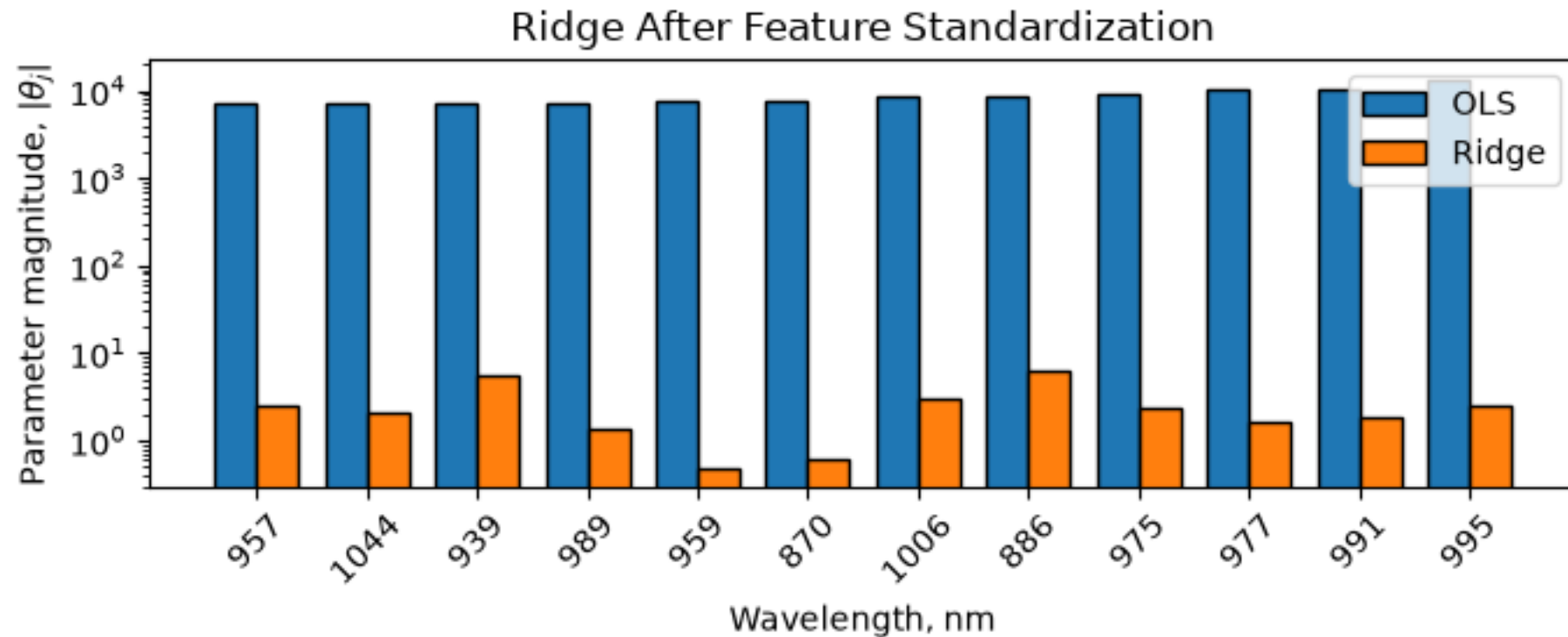
```
from sklearn.preprocessing import StandardScaler  
  
scaler = StandardScaler()  
  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

After standardization, one unit means one standard deviation for every feature.



The Impact of Standardization on Ridge Regression

Model	Training RMSE	Test RMSE
OLS	1.00	4.47
Ridge	3.24	4.25
Ridge after standardization	3.02	4.10



The Impact of Alpha on Ridge Shrinkage

$$\hat{\boldsymbol{\theta}}_{\text{ridge}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \left[\frac{1}{2n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 + \frac{\lambda}{2} \|\boldsymbol{\theta}\|_2^2 \right]$$

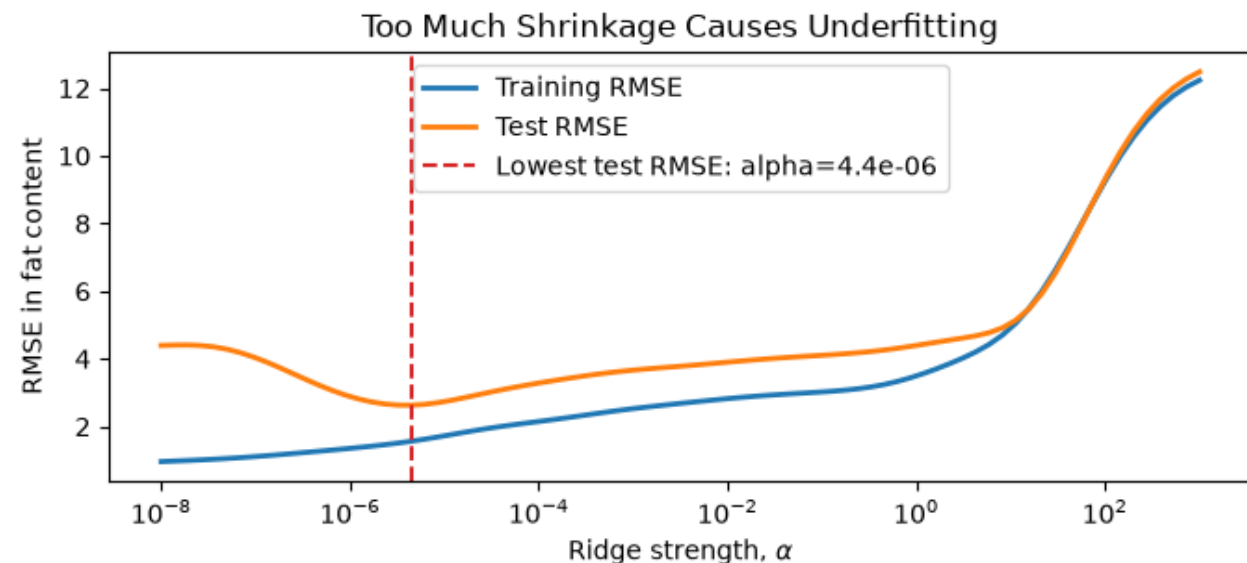
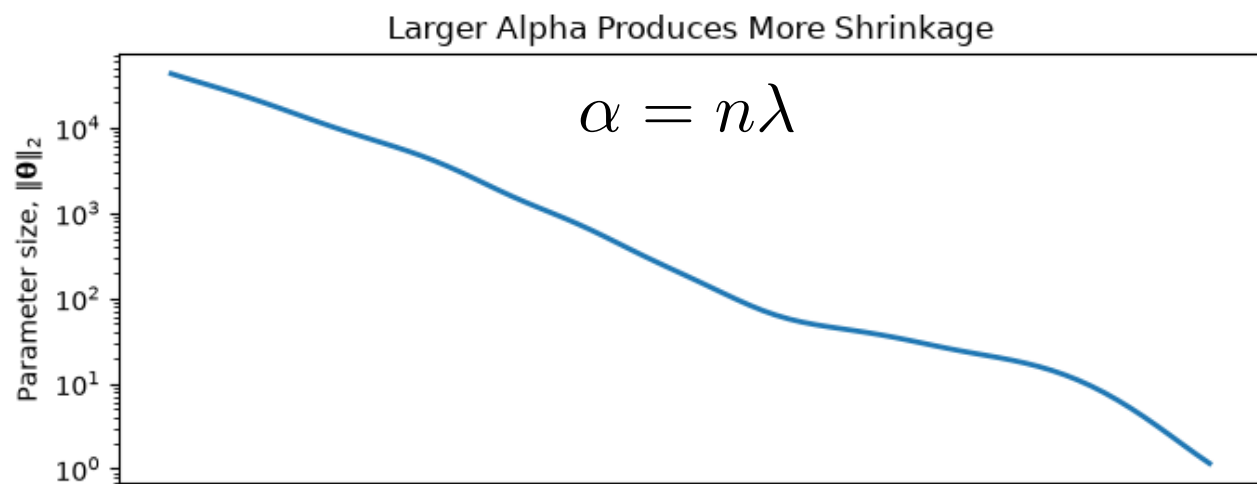
$\lambda = 0 \implies$ ordinary least squares

$$\lambda \uparrow \implies \left| \hat{\theta}_j \right| \downarrow$$

$$\lambda \rightarrow \infty \implies \hat{\boldsymbol{\theta}} \rightarrow \mathbf{0}$$

- λ too small: possible overfitting.
- λ too large: underfitting

Choose λ using validation data or cross-validation (not the test set).



Lasso Can Remove Features Completely

$$\hat{\boldsymbol{\theta}}_{\text{lasso}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \left[\frac{1}{2n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_1 \right] \quad z_j = \frac{1}{n} \mathbf{x}_j^\top \mathbf{y}$$

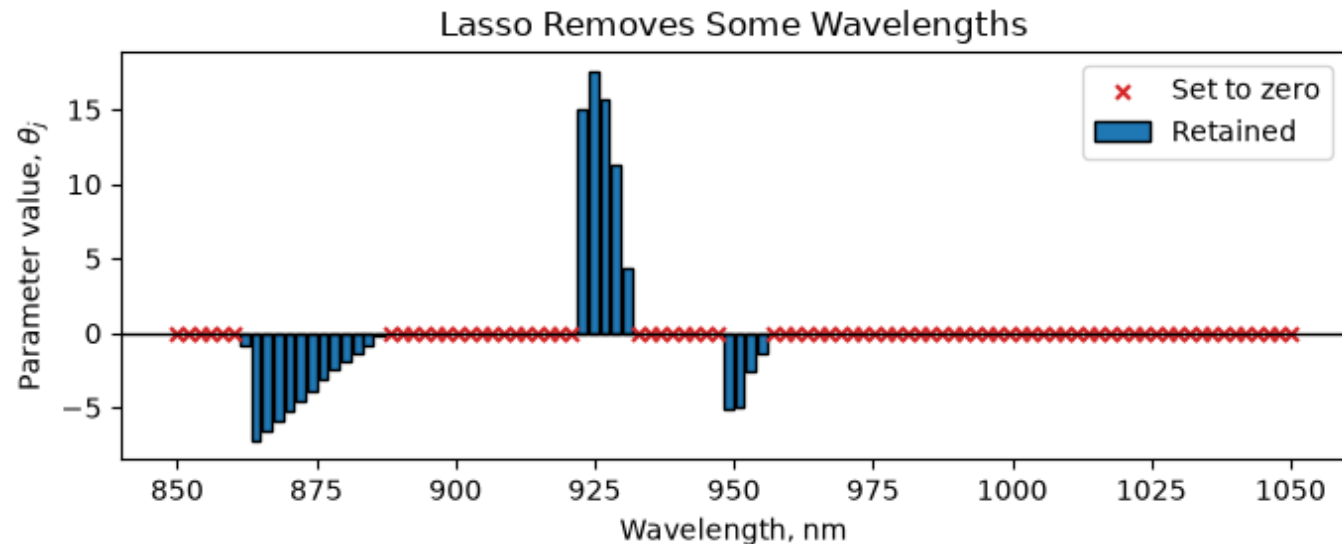
$$\|\boldsymbol{\theta}\|_1 = \sum_{j=1}^p |\theta_j| \quad \Rightarrow \quad \hat{\theta}_j = \operatorname{sign}(z_j) \max(|z_j| - \lambda, 0) \quad \Rightarrow \quad |z_j| \leq \lambda \quad \Rightarrow \quad \hat{\theta}_j = 0$$

```
class sklearn.linear_model.Lasso(alpha=1.0, *, fit_intercept=True,  
precompute=False, copy_X=True, max_iter=1000, tol=0.0001, warm_start=False,  
positive=False, random_state=None, selection='cyclic')
```

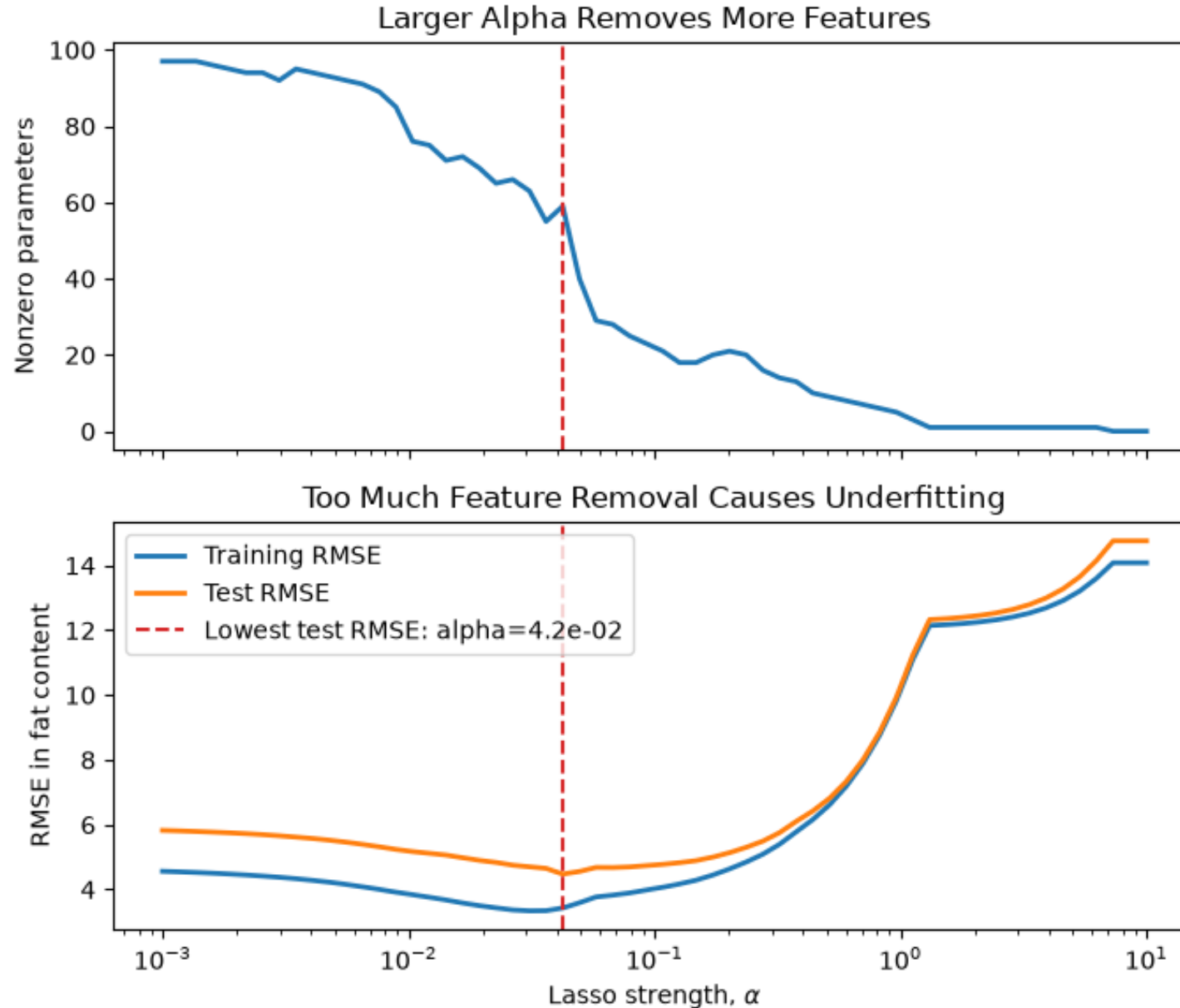
[\[source\]](#)

```
lasso = Lasso(alpha=0.1)  
lasso.fit(X_train_scaled, y_train)  
  
theta_lasso = lasso.coef_  
nonzero = np.abs(theta_lasso) > 1e-8
```

Ridge makes weak features smaller. Lasso can remove them completely.



The Impact of Alpha on Lasso Feature Selection



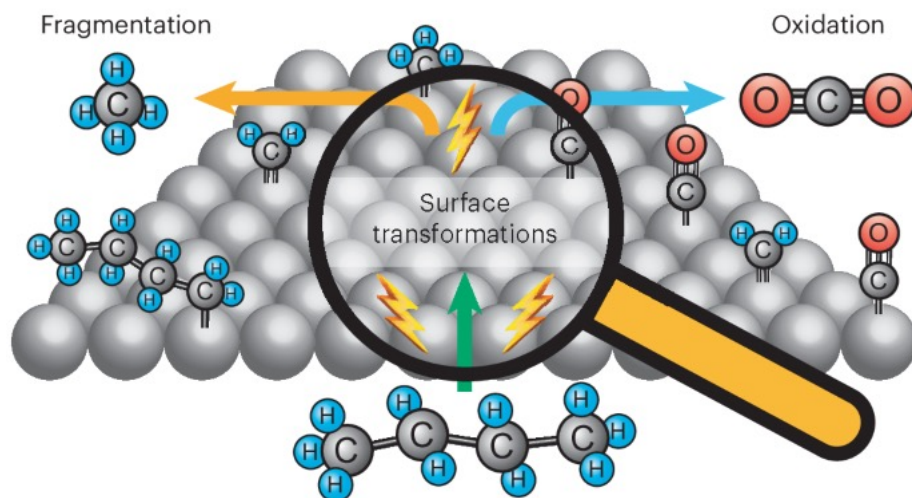
Lasso Estimates Gas Fluxes from Overlapping m/z Signals

Article | Published: 29 August 2024

Understanding the interplay between electrocatalytic C(sp³)-C(sp³) fragmentation and oxygenation reactions

Christine Lucky, Shengli Jiang, Chien-Rung Shih, Victor M. Zavala & Marcel Schreier 

Nature Catalysis 7, 1021–1031 (2024) | [Cite this article](#)

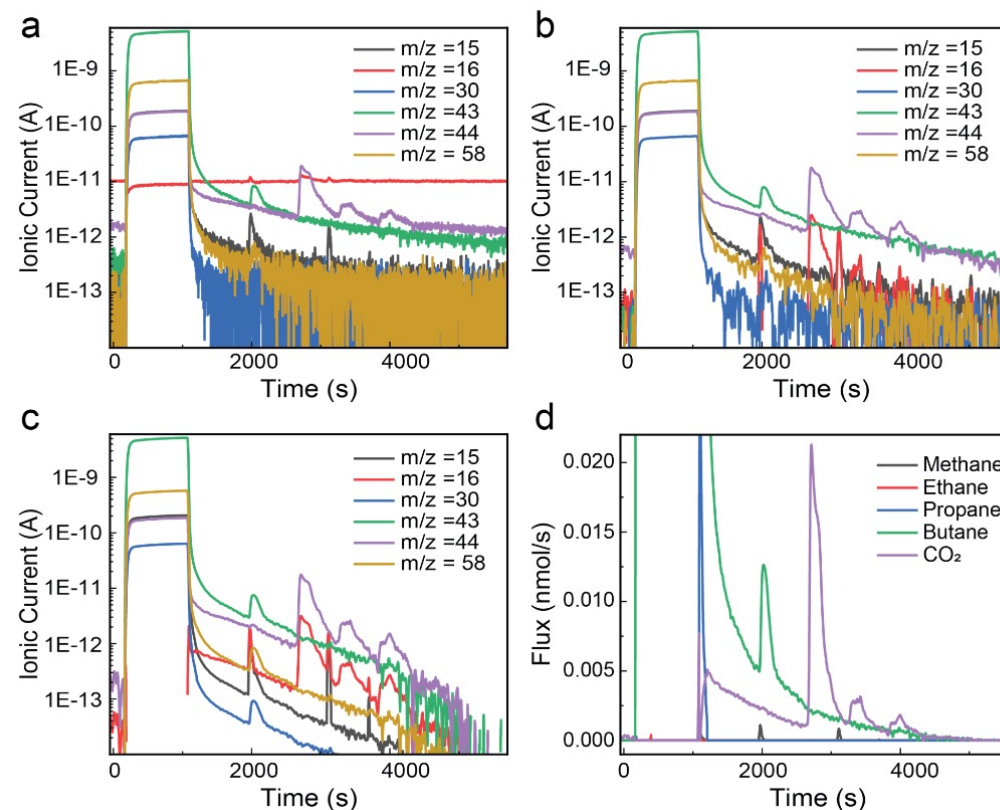


At each time point, we sought the solution to an optimization problem that could be expressed as a constrained least-squares problem with ℓ_1 normalization:

$$\min_w \|Aw - x\|_2^2 + \lambda \|w\|_1 \quad (2)$$

subject to $w_i \geq 0$ for $i = 1, \dots, 5$

where $w \in \mathbb{R}^5$ represents the flux of each gas. We aimed to minimize the difference between the predicted spectrum Aw and the observed spectrum x . To avoid overfitting and promote accurate quantification



A fitted parameter becomes smaller after applying Ridge. Does that mean the feature became less physically important?

- A. Yes
- B. No
- C. Only when the parameter becomes zero
- D. Only when the training error increases

Answer: **B.** Regularization changes how the model distributes predictive weight. A smaller fitted parameter is not evidence that the physical mechanism became weaker.

What can we conclude from the wavelengths retained by Lasso?

- A. They are the only chemically important wavelengths
- B. They cause changes in fat content
- C. They form a useful sparse set for prediction
- D. Every removed wavelength contains no information

Answer: **C.** Lasso selects a compact predictive set, but correlated features can often replace one another.

Hyperparameters Are Choices Made Outside Model Fitting

$$\hat{\boldsymbol{\theta}}_{\text{ridge}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \left[\frac{1}{2n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 + \frac{\lambda}{2} \|\boldsymbol{\theta}\|_2^2 \right] \quad \hat{\boldsymbol{\theta}}_{\text{lasso}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \left[\frac{1}{2n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_1 \right]$$

The screenshot shows the Merriam-Webster website with the search term 'hyper'. The left sidebar contains navigation links: Dictionary, Thesaurus, Games & Quizzes, and Word of the Day. The main content area displays the definition for 'hyper' as an adjective, with a pronunciation guide 'hy·per' and a list of synonyms including 'HIGH-STRUNG, EXCITABLE'. Below this, the prefix 'hyper-' is defined with examples like 'hypermarket', 'hypersensitive', 'hyperemia', 'hyperspace', and 'hypertext'.

options that we have/choices that we make
external to the process of finding model
parameters are often referenced as

hyperparameters

λ

in our prior examples:

regularization strength, the step-size in gradient descent,...

hyperparameter optimization – a process of that
should facilitate our selection of such things

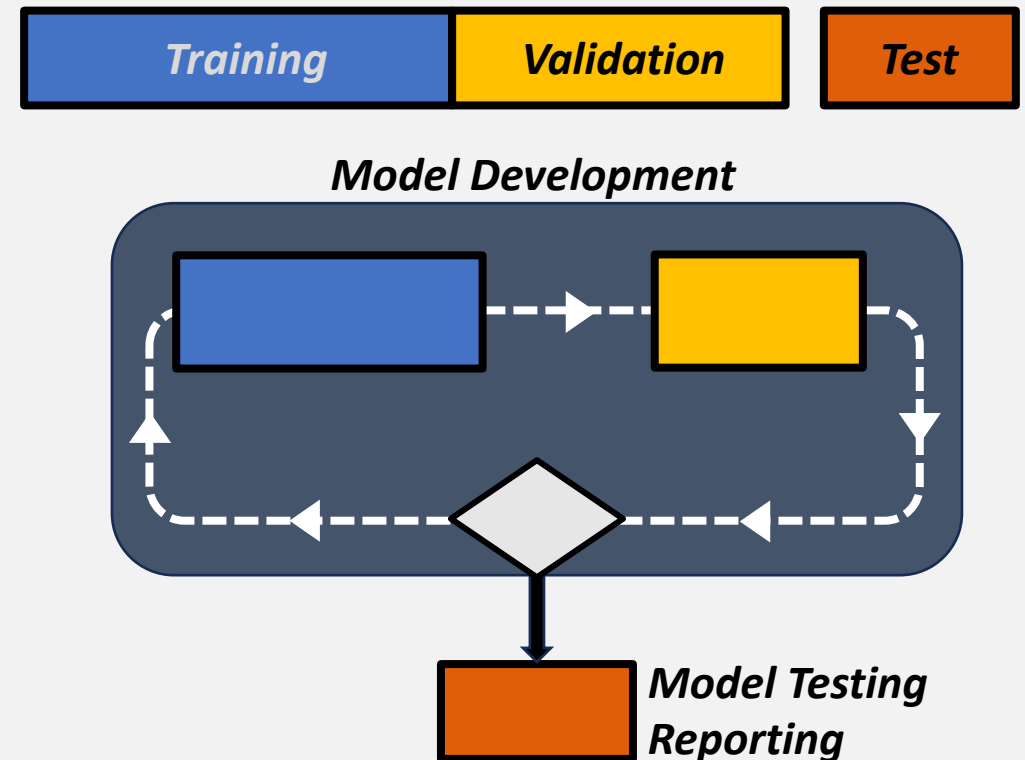
Training, Validation, and Test Sets Have Different Jobs

A primary goal of many machine learning regression problems is to achieve models that permit **generalization** (evaluation outside the domain of our dataset)

Cross-validation and/or the use of **train/validation/test splits** are two common strategies to mitigate overfitting a model to example data/inhibiting generalization

Basic premise of train-validation-test split

1. Shuffle dataset *randomly*
2. *Partition* your data into three groups, which we refer to as a **training** set, **validation** set, and a **test** set
3. How do we use these splits?
 - i. **training** – model is allowed to see this data and adjust its **parameters** accordingly
 - ii. **validation** – model does not adjust its **parameters** in response to this data but evaluation on the data provides feedback on generalizability for the given model type and mode of optimization
 - iii. **test** – model neither adjusts its parameters, model type, or mode of optimization – this is data that allows for critical assessment



The Best Data Split Depends on the Problem

A primary goal of many machine learning regression problems is to achieve models that permit **generalization** (evaluation outside the domain of our dataset)

Cross-validation and/or the use of **train/validation/test splits** are two common strategies to mitigate overfitting a model to example data/inhibiting generalization

Basic premise of train-validation-test split

1. Shuffle dataset *randomly*
2. *Partition* your data into three groups, which we refer to as a **training** set, **validation** set, and a **test** set
3. How do we use these splits?
 - i. **training** – model is allowed to see this data and adjust its **parameters** accordingly
 - ii. **validation** – model does not adjust its **parameters** in response to this data but evaluation on the data provides feedback on generalizability for the given model type and mode of optimization
 - iii. **test** – model neither adjusts its parameters, model type, or mode of optimization – this is data that allows for critical assessment



What is the right ratio of train : validation : test?

There is no single answer. Keep in mind...

- **how much data do you have?**
if data is precious, you may need to lean on more training data-without it, the model may just be garbage
- **how many hyperparameters do you have?**
you may need more on the validation side to practically guide model selection
- **what are you using your model for?**
either large or small test might be appropriate here.

Common Starting Points for a Three-Way Split

A primary goal of many machine learning regression problems is to achieve models that permit **generalization** (evaluation outside the domain of our dataset)

Cross-validation and/or the use of **train/validation/test splits** are two common strategies to mitigate overfitting a model to example data/inhibiting generalization

Basic premise of train-validation-test split

1. Shuffle dataset *randomly*
2. *Partition* your data into three groups, which we refer to as a **training** set, **validation** set, and a **test** set
3. How do we use these splits?
 - i. **training** – model is allowed to see this data and adjust its **parameters** accordingly
 - ii. **validation** – model does not adjust its **parameters** in response to this data but evaluation on the data provides feedback on generalizability for the given model type and mode of optimization
 - iii. **test** – model neither adjusts its parameters, model type, or mode of optimization – this is data that allows for critical assessment



What is the right ratio of train : validation : test?

As a rough guide, some standards are usually

- 50-80% for **training** data
- 10-30% for **validation** data
- 10-30% for **test** data

You will thus see

- We performed an 80/10/10 split
- ... 70/20/10
- ... 60/20/20
- etc.

**You try 30 values of α and choose the one with the lowest test RMSE.
What is the problem?**

- A. Ridge cannot use 30 values
- B. The test set has become part of model selection
- C. Training error must always equal test error
- D. The features were not standardized

Answer: **B.** Once the test set helps choose α , it is acting like validation data. The reported test performance is no longer a clean final evaluation.

After choosing α using the validation set, what should we do?

- A. Fit the final model using only the original training set
- B. Choose α again using the test set
- C. Refit the scaler and model using training + validation data, then evaluate the test set once
- D. Fit the scaler using only the test set

Answer: **C.** The validation set has completed its model-selection job. We can now combine training and validation data, refit using the chosen α , and use the untouched test set for the final evaluation.

Interactive Python Time

08_regularization.ipynb